

Autorización

Concepto

La **autorización** es el proceso de determinar si un usuario autenticado (cuya identidad ya ha sido verificada) tiene el permiso de realizar una acción específica o acceder a un recurso determinado.

Luego del proceso de **autenticación** (mediante el cual el sistema sabe quien es el usuario actual), se puede realizar el proceso de **autorización** (mediante el cual el sistema sabe qué acciones puede realizar el usuario).

¿Por qué usamos autorización?

En todo sistema que cuenta con usuarios, la autorización debe ser un proceso implementado obligatoriamente por los programadores. Su implementación tiene los siguientes fundamentos:

- **Principio del mínimo privilegio:** Este principio de seguridad establece que un usuario o proceso solo debe tener los permisos necesarios para realizar sus tareas. La autorización asegura que un usuario no pueda acceder a datos o funciones que no le corresponden, como ver la información de otros usuarios o eliminar registros críticos del sistema.
- **Control de acceso:** La autorización es el mecanismo que implementa el control de acceso. Permite que el administrador de un sistema configure quién puede acceder a qué elementos del sistema, en base a roles, permisos o reglas específicas. Esto es importante en sistemas multiusuario, como redes sociales, aplicaciones de gestión de contenido o plataformas empresariales, donde diferentes tipos de usuarios (administradores, editores, usuarios normales) necesitan distintos niveles de acceso.
- **Protección de datos sensibles:** Sin autorización, un atacante que logre autenticarse podría acceder a información confidencial. La autorización previene este tipo de escalada de privilegios, asegurando que solo los usuarios con la autorización adecuada puedan ver o modificar datos sensibles. Por ejemplo, en un sistema bancario, la autorización previene que un cliente pueda ver el saldo de la cuenta de otro cliente.
- **Experiencia de usuario:** La autorización no solo es una medida de seguridad, sino que también mejora la experiencia del usuario al personalizar la interfaz. Al saber qué puede hacer un usuario, la aplicación puede ocultar botones o menús que no son relevantes para su rol, evitando errores y confusión. Por ejemplo, si un usuario no tiene permiso para crear una publicación, el botón "Crear nueva" no se mostrará.

Estrategias de autorización

Para implementar este mecanismo en un sistema, podemos optar por diferentes estrategias, entre las cuales podemos encontrar las siguientes:

RBAC

El **Control de Acceso Basado en Roles** (y Permisos) permite definir que cada usuario tenga un rol, el cual a su vez consistirá en una serie de permisos que defina las acciones posibles en el sistema. También es posible asignar directamente el permiso al usuario, sin usar el rol como intermediario.

Es una propuesta sólida, de gran flexibilidad, y es la que usaremos en esta cátedra.

ABAC

El **Control de Acceso Basado en Atributos** es una estrategia más granular y flexible. En lugar de asignar permisos a un rol, se evalúan un conjunto de atributos para determinar si una acción está permitida, los cuales pueden ser:

- **Atributos del usuario:** como su edad, departamento, cargo, o la hora del último inicio de sesión.
- **Atributos del recurso:** como el tipo de archivo (público o privado), su fecha de creación o su autor.
- **Atributos del entorno:** como la hora del día, la dirección IP desde la que se realiza la petición o la ubicación geográfica.
- **Atributos de la acción:** como leer, escribir o eliminar.

Combinando estos atributos, se generan políticas que permiten definir el acceso. Un ejemplo de política podría ser: *"Un usuario con el rol de Editor puede editar un documento si y solo si el documento está en el estado Borrador y el usuario es el autor del mismo, y si la petición se realiza durante el horario laboral"*.

ReBAC

El **Control de Acceso Basado en Relaciones** (o control de acceso relacional) es una estrategia que se centra en las relaciones que existen entre los usuarios y los recursos, de modo que un usuario poseerá permisos para interactuar solamente con los recursos con los cuales posee un "parentesco". Un ejemplo de política de este tipo podría ser: *"Un usuario puede ver un proyecto si el usuario es miembro de ese proyecto"*.

Conceptos adicionales necesarios

Roles

Un **rol** es un conjunto predefinido de permisos o responsabilidades asignadas a un usuario. Por lo general, los usuarios de un sistema se agrupan en roles. Por ejemplo, en un blog, puedes tener los roles de **"Administrador"** (que puede crear, editar y eliminar publicaciones de cualquier usuario), **"Editor"** (que puede editar sus propias publicaciones y las de otros, pero no eliminarlas) y **"Usuario"** (que solo puede ver las publicaciones y comentar). Los roles simplifican la gestión de permisos al permitir que los cambios se apliquen a un grupo de usuarios a la vez.

Permisos

Un **permiso** es la unidad más granular de autorización. Es una capacidad o derecho específico para realizar una acción, como "**crear publicación**", "**editar publicación**" o "**eliminar usuario**". Un rol puede tener múltiples permisos. Por ejemplo, el rol de "Administrador" podría tener los permisos crear publicacion, editar publicacion y eliminar publicacion. La combinación de roles y permisos es la base de la mayoría de los sistemas de control de acceso basados en roles (RBAC).

Gates y Políticas (En Laravel)

Un **Gate** (o "puerta") es una función de cierre (closure) que recibe como argumento un usuario y los argumentos adicionales necesarios para determinar si el usuario tiene permiso para realizar una acción. Los Gates son ideales para permisos simples y acciones que no están directamente ligadas a un modelo Eloquent específico.

En el siguiente ejemplo, se define un Gate llamado "*edit-settings*" que devolverá true si el usuario actual es administrador del sistema o no.

```
use Illuminate\Support\Facades\Gate;
use App\Models\User;

Gate::define('edit-settings', function (User $user) {
    return $user->isAdmin();
});
```

Una **Policy** (o "política") es una clase que agrupa la lógica de autorización para un **modelo específico**. Las Políticas son el método preferido de Laravel cuando se necesita controlar el acceso a recursos de un modelo (ej. una publicación, un usuario, un comentario). Cada método en una Policy corresponde a una acción que se puede realizar sobre ese modelo, por ejemplo: **view** (ver), **create** (crear), **update** (actualizar), **delete** (eliminar).

Para crear una Policy usamos el siguiente comando:

```
php artisan make:policy PostPolicy --model=Post
```

Una vez creada la clase, podemos editar el método específico de la policy que deseamos controlar. Por ejemplo, si queremos indicar que un usuario puede editar un Post solamente si es el usuario que creó dicho recurso, completaríamos el código de la siguiente forma:

```
use App\Models\User;
use App\Models\Post;
```

```
class PostPolicy
```

```

{
    public function update(User $user, Post $post): bool
    {
        // Un usuario puede actualizar una publicación si es el autor
        de la misma
        return $user->id === $post->user_id;
    }
}

```

Como usar las Policies

Laravel proporciona un método llamado **can()** que permite preguntar si un usuario puede realizar una acción sobre un modelo específico:

```

if ($user->can('update', $post)) {
    // El usuario puede actualizar la publicación
}

```

De la misma forma, contamos con la directiva de Blade **@can** que, en caso de cumplirse, mostrará u ocultará parte de la página si se cumple la política indicada:

```

@can('update', $post)
    <a href="/posts/{{ $post->id }}/edit">Editar publicación</a>
@endcan

```

Ejemplo: restringir acceso a recursos según pertenencia

Tomemos el siguiente ejemplo: tenemos un sistema con posts que son creados por diferentes usuarios y queremos establecer que un post solamente puede ser editado por el usuario que lo creó. Para ello tomaremos los siguientes pasos.

1) Creamos el PostPolicy

```
php artisan make:policy PostPolicy --model=Post
```

2) Definir la lógica en el método update()

```

public function update(User $user, Post $post): bool
{
    return $user->id === $post->user_id;
}

```

```
}
```

3) Aplicar la comprobación en el controlador

Asumiendo que tenemos un PostController, vamos a aplicar el uso de la Policy en los métodos edit y update del controlador utilizando el método **authorize()**. Este método automáticamente toma el usuario que inició sesión (por ello no se puede utilizar en rutas o controladores de recursos públicos) y lo comprueba en base a una Policy establecida; si falla la comprobación se arrojará un error 403.

```
class PostController extends Controller
{

    public function edit(Post $post)
    {
        // 1. COMPROBACIÓN DE AUTORIZACIÓN:
        $this->authorize('update', $post);
        // Resto del codigo...
    }

    public function update(Request $request, Post $post)
    {
        // 2. COMPROBACIÓN DE AUTORIZACIÓN:
        $this->authorize('update', $post);
        // Resto del código...
    }
}
```

Spatie Laravel Permission

Debido a que la autorización es un tópico muy común, existen varias librerías de la comunidad que permiten implementar este mecanismo en una aplicación de Laravel. Utilizaremos la más popular que se denomina **Spatie/Laravel Permission**.

Github: <https://github.com/spatie/laravel-permission>

Documentación: <https://spatie.be/docs/laravel-permission>

Instalación

Para instalarlo en nuestro proyecto ingresamos el siguiente comando en nuestra consola:

```
composer require spatie/laravel-permission
```

Luego podemos publicar las migraciones y el archivo de configuración:

```
php artisan vendor:publish --  
provider="Spatie\Permission\PermissionServiceProvider"
```

El siguiente paso consiste en limpiar la cache de configuración del proyecto:

```
php artisan config:clear
```

Ahora es momento de ejecutar las migraciones para crear las tablas necesarias en nuestra base de datos:

```
php artisan migrate
```

Y por último, modificaremos el modelo User para agregar el trait HasRoles:

```
use Spatie\Permission\Traits\HasRoles;  
  
class User extends Authenticatable  
{  
    use HasFactory, Notifiable, HasRoles;  
    // Resto del código...  
}
```

¿Cómo funciona esta librería?

Al ejecutar la migración se han instalado 5 tablas:

- **permissions:** esta es la tabla que almacena todos los permisos únicos que existen en la aplicación. Se sugiere que cada permiso tenga un nombre que combine la acción con el recurso específico y que se escriba en minúsculas y con un guion medio. Por ejemplo: *edit-post* para editar un post, *view-post* para ver un post, etc.
- **roles:** es la tabla que especifica los roles de la aplicación. Por ejemplo: *admin*, *editor*, etc.
- **role_has_permissions:** tabla pivot entre las dos tablas anteriores. Permite asignar permisos a un rol específico.
- **model_has_roles:** tabla pivot entre “roles” y cualquier otro modelo de nuestra aplicación (por lo general con la tabla “users”). Esta tabla permite definir que registros o usuarios posee cuales roles en la aplicación. Por ejemplo: mi usuario “Juan” puede tener asignado el rol “admin”, y junto con él todos los permisos asignados al rol.

- **model_has_permissions:** tabla pivot entre “permissions” y cualquier otro modelo de nuestra aplicación (por lo general con la tabla “users”). Esta tabla permite definir que registros o usuarios poseen ciertos permisos específicos. Por más que un usuario tenga asignado un rol con sus respectivos permisos, podemos asignar permisos a ciertos usuarios si lo consideramos necesario. Por ejemplo: si mi usuario “Gloria” tiene el rol “editor”, aun así le puedo asignar el permiso “deshabilitar-usuario” para que me ayude a deshabilitar los usuarios de la aplicación.

Con esta estructura, ahora podemos comprobar si un usuario posee roles o permisos de la siguiente forma:

```
use App\Models\User;
```

```
$user = User::find(1);
// Comprobar si el usuario tiene un rol
if ($user->hasRole('admin')) {
    // ...
}
// Comprobar si el usuario tiene un permiso directo o a través de un rol
if ($user->can('create-post')) {
    // ...
}
```

NOTA: en este caso estamos buscando un usuario de la BD, pero también podemos obtener el usuario actual mediante los métodos vistos en la clase pasada (revisar el apunte).

También podemos condicionar los elementos de la vista con las siguientes directivas de Blade:

```
@role('admin')
    <a href="/dashboard/settings">Ver ajustes de administrador</a>
@endrole

@can('create-post')
    <a href="/posts/create">Crear nueva publicación</a>
@elsecan('edit-post')
    <p>Solo puedes editar publicaciones.</p>
@endcan
```

Crear nuestros roles y permisos

Antes de avanzar necesitamos que nuestra aplicación cuente con roles y permisos para poder asignar a los usuarios. exploremos dos alternativas

1) Usar Tinker

Si deseamos crear estos elementos de forma rápida podemos usar Tinker (visto en la clase pasada). Para ello desde la consola ejecutamos el comando:

```
php artisan tinker
```

Y ahora podemos crear los roles y permisos:

```
// Crear un rol 'Administrador'
$rol = Spatie\Permission\Models\Role::create(['name' => 'admin']);

// Crear un permiso para publicar un post
$permiso = Spatie\Permission\Models\Permission::create(['name' =>
'create-post']);

// Asignar el permiso 'create-post' al rol 'admin'
$rol->givePermissionTo($permiso);

// Crear un usuario de prueba
// Tambien podemos buscar uno ya existe con User::find(1);
$user = App\Models\User::factory()->create(['email' =>
'admin@example.com']);

// Asignar el rol 'admin' al usuario de prueba
$user->assignRole('admin');
```

2) Usar Seeders

Para asegurarnos que los roles y permisos persistan cada vez que realizamos una migración desde cero, podemos crear un seeder que se encargue de crear roles y permisos por defecto. Primero creamos el seeder con el siguiente comando:

```
php artisan make:seeder RolesPermisosSeeder
```


Luego modificamos el código de la clase de la siguiente forma:

```
namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Spatie\Permission\Models\Role;
use Spatie\Permission\Models\Permission;
use App\Models\User;

class RolesPermisosSeeder extends Seeder
{
    public function run(): void
    {
        // Reinicia los permisos en la caché
        app()[\Spatie\Permission\PermissionRegistrar::class]-
        >forgetCachedPermissions();

        // Crear permisos
        $createPostPermission = Permission::create(['name' => 'create-
post']);
        $editPostPermission = Permission::create(['name' => 'edit-
post']);
        $deletePostPermission = Permission::create(['name' => 'delete-
post']);

        // Crear roles y asignar permisos
        $adminRole = Role::create(['name' => 'admin']);
        $editorRole = Role::create(['name' => 'editor']);
        $viewerRole = Role::create(['name' => 'viewer']);

        $adminRole->givePermissionTo([
            $createPostPermission,
            $editPostPermission,
            $deletePostPermission,
```

```

]);

$editorRole->givePermissionTo([
    $createPostPermission,
    $editPostPermission,
]);

// Asignar roles a usuarios de prueba
$adminUser = User::factory()->create(['name' => 'Admin User',
'email' => 'admin@example.com']);
$adminUser->assignRole('admin');

$editorUser = User::factory()->create(['name' => 'Editor
User', 'email' => 'editor@example.com']);
$editorUser->assignRole('editor');

// Si quiero asignar un permiso unico al usuario
$editorUser->givePermissionTo('delete-post');
}
}

```

Finalmente, recordemos agregar el llamado a este Seeder desde el archivo DatabaseSeeder en su método run():

```

public function run(): void
{
    // Ejecutar todos los seeders una sola vez
    $this->call([
        RolesPermisosSeeder::class,
        // Otros Seeders...
    ]);
}

```

Middlewares

Un **middleware** en Laravel es una capa de filtrado que se ejecuta antes de que una petición HTTP llegue a su destino final (la ruta o el controlador) o después de que la respuesta sea generada.

De forma práctica, los usamos para controlar si una petición tiene una condición previa que cumplir, como por ejemplo:

- Que el usuario esté autenticado
- Que el usuario posea ciertos rol o permiso
- Que en una petición exista el token CSRF (comportamiento por defecto de los formularios)
- Que se guarde la actividad del usuario en un archivo (logging)
- Restringir la actividad de un usuario por comportamiento sospechoso

Por lo general estas condiciones se aplican a nivel de rutas, pero también es posible utilizarlos en los controladores.

¿Como funciona un Middleware?

El flujo de una petición en Laravel con middleware es el siguiente:

1. **Petición entrante:** Un usuario envía una petición a una ruta de tu aplicación.
2. **Middleware:** Antes de que la petición llegue al controlador, pasa a través de una cadena de middlewares. Cada uno de estos puede realizar una acción:
 - **Permitir el paso:** El middleware puede dejar que la petición continúe al siguiente eslabón de la cadena.
 - **Redireccionar:** El middleware puede redirigir al usuario a otra página.
 - **Detener la petición:** El middleware puede detener la petición por completo y devolver una respuesta inmediata.
3. **Ejecución del controlador:** Si la petición pasa todos los middlewares, llega al controlador, que ejecuta la lógica de la aplicación y devuelve una respuesta.
4. **Middleware (después de la respuesta):** La respuesta del controlador puede pasar de nuevo por los middlewares en orden inverso, permitiendo que se realicen acciones después de que la lógica de la aplicación se haya ejecutado.

Veamos ahora algunos escenarios comunes en los cuales podemos usar middlewares:

a) Restringir acceso al dashboard (u otras rutas)

El Dashboard de una aplicación solo debería ser visible a los usuarios autenticados. Para cumplir esta condición usamos el middleware **auth** sobre la ruta del dashboard.

```
Route::get('/dashboard', function() {
```

```
        echo "Bienvenido al dashboard (esta zona es privada)";
    })->middleware('auth')->name('dashboard');
```

De esta forma, si un usuario no está autenticado no podrá ingresar a dicha ruta.

b) Permitir el acceso solamente a las visitas

En el caso de que una ruta no deba ser accesible a los usuarios autenticados, podemos usar el middleware **guest** para asegurarnos que el usuario sea una visita. Un claro ejemplo es la página para iniciar sesión, la cual no tiene sentido que la pueda ver un usuario que ya cuenta con una sesión.

```
Route::get('/login', [AuthController::class, 'showLogin'])
    ->name('login')
    ->middleware('guest');
```

c) Restringir acceso a una ruta según un rol

Si usamos el paquete Laravel Permission tenemos acceso al middleware **rol**: que permite restringir el acceso a una ruta en base a un rol puntual. Si el usuario no cuenta con dicho rol, no podrá visitar la ruta.

```
Route::get('/admin/settings', function () {
    return 'Panel de administración.';
})->middleware(['role:admin']);
```

También podemos combinar condiciones con el carácter **|** si una ruta admite el acceso a más de un rol.

```
Route::get('/dashboard', function () {
    return 'Panel de control.';
})->middleware(['role:admin|editor']);
```

d) Restringir acceso a una ruta según un permiso

De la misma forma, el paquete Laravel Permission ofrece el middleware **permission**: para restringir el acceso en base a uno o varios permisos.

```
Route::get('/posts/create', function () {
    return 'Crear una publicación.';
})->middleware(['permission:create-post']);
```

```
Route::get('/dashboard/posts', function () {  
    return 'Ver publicaciones.';  
})->middleware(['permission:create-post|edit-post']);
```

Pro tip: agrupar rutas con middlewares

Llegará el momento de restringir todas las páginas “privadas” de la aplicación con el middleware auth, pero en lugar de repetir el mismo en cada ruta, podemos crear un grupo de rutas que utilicen este middleware, y dentro del mismo ubicar todas las rutas que componen la zona “privada” de la aplicación:

```
Route::middleware(['auth'])->group(function () {  
    Route::get('/dashboard', function () {  
        return '¡Bienvenido a tu Dashboard!';  
    })->name('dashboard');
```

```
    Route::get('/profile', function () {  
        return 'Página de perfil del usuario.';  
    })->name('profile');
```

```
    Route::get('/settings', function () {  
        return 'Página de configuración.';  
    })->name('settings');
```

```
    Route::post('/logout', [AuthController::class, 'logout'])->  
        name('logout');  
    // Otras rutas...  
});
```